
UNIT 4: COMBINING AND RESHAPING DATASETS

Structure	Page No.
4.0 Introduction	93
4.1 Objectives	93
4.2 Indexing	93
4.3 Hierarchical Indexing	96
4.3.1 Reordering Hierarchical Levels	99
4.3.2 Sorting Levels	100
4.4 Combining and Merging Datasets	101
4.4.1 Merging	101
4.4.2 Concatenating	104
4.4.3 Combining Data with Overlap	106
4.5 Reshaping	109
4.5.1 Pivoting	109
4.5.2 <code>stack()</code> and <code>unstack()</code> methods	110
4.5.2 Reshaping with Hierarchical Indexing	111
4.6 Summary	112
4.7 Solutions/Answers	113

4.0 INTRODUCTION

In the previous units of this course, you have gone through the concepts of the basics of data wrangling, including data cleaning and data preparation. This was followed by a discussion on data transformations to different forms using various methods, such as discretisation and string manipulation. This unit focuses on techniques for combining data. In addition, the unit also discusses the concept of data reshaping. One of the key topics discussed in the context of data reshaping is hierarchical indexing. Since indexing the data is an important concept, we briefly outlined the techniques and concepts of indexing, which was followed by a discussion on Hierarchical indexing. The unit also discusses the concept of sorting data, pivoting of data. Further, it highlights the techniques of combining data, including merging and concatenation of data. The unit also presents the Python programs related to these topics.

4.1 OBJECTIVES

After going through this unit, you should be able to:

- Define the use of indexing in data frames
- Create simple indexes using Python
- Create hierarchical level indexes, as per the needs of an application
- Describe the process of merging and concatenating data sets
- Write Python code to merge or concatenate data sets
- Discuss methods of reshaping the data.

4.2 INDEXING

The basic concept of indexing has been explained to you in the course MCS-207, Unit 4. Indexing requires you to create a key, which may be primary or secondary, that is part of each record or can be created separately, with the basic objective of enhancing the speed of query processing. Please note that indexing can enhance the speed of only

those queries that use that index. For example, consider the following data of an organisation:

Index #	Name	Department	Specialisation	Salary
0	Arav S	Design	Web Development	100000
1	Ravi M	Database	SQL	200000
2	Rehan D	Design	SQL	150000
3	Ben A	Database	Web Development	200000
4	Rai Y	Design	SQL	100000

Figure 1: List of Employees

Consider you need to list data in alphabetical order of names in several applications, then you can create an index on the **Name** attribute of the table, thus listing the names along with the associated index # as follows:

Name	Index #
Arav S	0
Rai Y	4
Ben A	3
Ravi M	1
Rehan D	2

Figure 2: Index on Name (assume names are unique)

The size of the index, as shown in Figure 2, is smaller than the data set of Figure 1. Indexes help in faster searches. For example, in case you are trying to find the salary of person whose name is “Rai Y”, then if there are no indexes then you need to search almost the entire data set, whereas since the indexes are sorted on the attribute value, therefore, you search to find the “Rai Y” in the index will be much faster (for example, you may use binary search in case the complete index is in the memory). Once you locate the stated value of *Name* attribute in the index, you will be able to find the Index #, which will help you in locating the actual record.

You can also create indexes on other attributes, if needed. For example, you may create another index on the **Department** attribute. Such an index, as shown below, may look slightly different from the index shown above.

Department	Index #
Database	1, 3
Design	0, 2, 4

Figure 3: Index on Department (a clustering index)

Why Indexing?

The main advantages of Indexing can be stated as:

1. **Enhancement in Search Performance:** One of the major advantages of indexing is that it enhances the performance of information retrieval. For example, if you are finding the list of employees of the *Database* department, and there is no index, then you need to go through the entire data or set of records to answer that query. However, if an index as shown in Figure 3 is maintained, then the data of the employees of the “Database” department can be determined with almost no time.
2. **Ease of Merging of datasets using an index:** Several applications require data to be combined or merged from various sources or data sets using some criteria. This process of merging can be made faster if the data sets to be merged are indexed on the same attribute related to the criteria.
3. **Accessing the Uniqueness:** In Figure 2, we have assumed that the index is unique. This will ensure that you cannot enter the records of the person with the same name for the second time. Such primary key indexes ensure that data duplicity is minimised.

The disadvantage of using indexes is that they increase the load of updating a data set, as along with the data set, you need to update all the related indexes made on that data set. For example, consider the Department of the employee named “Arav S” is changed from “Design” to “Database”, then this change would require a change in Index # 0 of the data set given in Figure 1, and also in the index as shown in Figure 3. In addition, indexes and data sets both require storage space.

Please note that a key decision in indexing is to decide what indexes are to be maintained for a data set. The overall objective is to facilitate search capability in a data set.

Creating an Index Using Pandas

Let us now demonstrate the process of creating indexes. For this purpose, we use Python and its Pandas library. The following code is created with comments so that you can understand the entire code.

```
#(a) Step 1: Import the Pandas library
import pandas as pd
#Step 2: Create the data Frame
Figure1 = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
'Rai Y'],
'Department': ['Design', 'Database', 'Design', 'Database',
'Design'],
'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
'Salary': [100000, 200000, 150000, 200000, 100000]}
employeedata = pd.DataFrame(Figure1)

#(b) A default integer index value is automatically defined by
pandas dataframe
# index starting from index 0, as shown in Figure 1. You can
access the data frame
# By using .ilocmethod.
print(employeedata.iloc[4])
print("\n")

# (c) Assigning index on a specific field, first set the Index
and then
# access the row by using the index value of the field ('Name')
Employeedata_Name_index = employeedata.set_index('Name')
print(Employeedata_Name_index.loc['Ben A'])
print("\n")

# We make a second Index on Department, it is a clustering
index
# access the row by using the index value of the field
('Department')
Employeedata_Department_index =
employeedata.set_index('Department')
print(Employeedata_Department_index.loc['Design'])
print("\n")

# (d) You can also use labels of your choice as index values.
This requires a change in #the declaration of the a DataFrame.
For example, we use a, b c, d, e as index values
Employeedata_label_index= pd.DataFrame(Figure1, index=['a',
'b', 'c', 'd', 'e'])
print(Employeedata_label_index.loc['e'])
print("\n")
```

Figure 4: The Program using Indexing

Figure 5 shows the result of execution of the code shown in Figure 4.

Name	Rai Y
Department	Design
Specialisation	SQL
Salary	100000
Name: 4, dtype: object	

Department	Database
Specialisation	Web Development
Salary	200000
Name: Ben A, dtype: object	

	Name	Specialisation	Salary
Department			
Design	Arav S	Web Development	100000
Design	Rehan D	SQL	150000
Design	Rai Y	SQL	100000

Name	Rai Y
Department	Design
Specialisation	SQL
Salary	100000
Name: e, dtype: object	

Figure 5: Result on execution of program of Figure 4

Please note the following points about the output:

1. Part (a) of Figure 4 creates a data frame in pandas. This data frame shows the data in Figure 1. First, a variable name Figure1 is used to create a data frame, which is then converted to a pandas data frame named **employee data** object.
2. In part (b) of Figure 4, the method .iloc was executed on the **employee data** object for the index value 4. Thus, resulting in the first object output in Figure 5. Please note this output is identical to Figure 1, index 4 object of the data frame.
3. Part (c) creates two indexes (i) the first index is created on **Name**, followed by printing of data of object having Name index value 'Ben A' using .loc method, which is printed next in Figure 5. (ii) The second index is created on **Department**, followed by printing of all the records of the **employee data** data frame having Department index value as 'Design'. You may observe that it prints three records where Department is Design. Please note that when making an index on a data set, the output shows the entire but indexed data frame.
4. At part (d) of Figure 4, a labelled index is created, which is used to print the object indexed by value 'e', which is the last output of Figure 5.

Now, you should be able to create indexes. Let us now discuss the hierarchical indexing in the next section.

4.3 HIERARCHICAL INDEXING

Hierarchical indexing is used to create several levels of indexes within the same data frame or data structure. These indexes are also sometimes referred to as multi-indexing. For example, in Figure 1, you may create hierarchical indexes on Department and Specialisation.

Why are they needed?

Hierarchical indexing is useful when a query on data uses data from multiple unrelated data attributes. They allow efficient data access and processing. For example, the hierarchical index on Department and Specialisation may allow efficient execution of the query: “List Name and Salary of employees in Database department who have SQL specialisation”, as data access using a hierarchical index on Department and

Specialisation would be much faster for this query. The hierarchical query also helps in efficient grouping of data.

Advantages of Hierarchical Indexing

The following are the advantages of hierarchical indexing:

- An interesting advantage of hierarchical indexing is that it allows data analysis, which involves multiple dimensions, even if the data is stored in a data frame.
- It helps in faster access to data and allows for grouping data more efficiently. Thus, enabling faster performance of aggregate operations on data.

Creating Hierarchical Index Using Pandas

The program given in Figure 6 is a program that demonstrates the use of a hierarchical index.

```
#Program 2: Part (a)
#Import the Pandas library
import pandas as pd
#Create the data Frame
Figure1 = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A', 'Rai Y'],
           'Department': ['Design', 'Database', 'Design', 'Database', 'Design'],
           'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web Development', 'SQL'],
           'Salary': [100000, 200000, 150000, 200000, 100000]}
employeedata = pd.DataFrame(Figure1)
```

```
#Part(b): Creating an index
Employeedata_multi_index =
employeedata.set_index(['Department', 'Specialisation'])
print("Part(b): Creating and Displaying Multi Index")
print(Employeedata_multi_index)
print("\n")
```

```
#Part (c): Sorting the indexes
print("Part(c): Sorting the Indexes for better results")
Employeedata_multi_index_sorted =
Employeedata_multi_index.sort_index()
print(Employeedata_multi_index_sorted)
print("\n")
```

```
#Part (d):Accessing data value in a Multi-index"
print("Part(d): Find the Salary and Name of employees in Database department with SQL specialisation")
print(Employeedata_multi_index_sorted.loc[('Database','SQL')])
print("\n")
```

```
# Part (e): Slicing
#Select employees of 'Department' category Slicing
print("Part(e): Find the data of the Database Department")
print(Employeedata_multi_index_sorted.loc['Database'])
print("\n")
# Select 'SQL' within 'Design' Slicing
print("Find Employees in Design department who know SQL")
print(Employeedata_multi_index_sorted.loc[('Design', 'SQL')])
print("\n")
```

```
# Part (f): Swapping Department with Specialisation (the index levels)
print("Part(f): Find all people who know SQL irrespective of department")
EmployeeData_multi_index_sortedandswapped =
EmployeeData_multi_index_sorted.swaplevel('Department',
'Specialisation')
print(EmployeeData_multi_index_sortedandswapped.loc['SQL'])
print("\n")
```

```
# Part (g): Grouping by Department to find the mean Salary
print("Part(g): Find mean Salary of employee of each department")
mean_salary =
EmployeeData_multi_index_sorted.groupby(level='Department').mean('Salary')
print(mean_salary)
print("\n")
```

Figure 6: Hierarchical Index and Operations

The output of this program is shown in Figure 7.

Part(b): Creating and Displaying Multi Index

Department	Specialisation	Name	Salary
Design	Web Development	Arav S	100000
Database	SQL	Ravi M	200000
Design	SQL	Rehan D	150000
Database	Web Development	Ben A	200000
Design	SQL	Rai Y	100000

Part(c): Sorting the Indexes for better results

Department	Specialisation	Name	Salary
Database	SQL	Ravi M	200000
	Web Development	Ben A	200000
Design	SQL	Rehan D	150000
	SQL	Rai Y	100000
	Web Development	Arav S	100000

Part(d): Find the Salary and Name of employees in Database department with SQL specialisation

Department	Specialisation	Name	Salary
Database	SQL	Ravi M	200000

Part(e): Slicing

Find the data of the Database Department

Specialisation	Name	Salary
SQL	Ravi M	200000
Web Development	Ben A	200000

Find Employees in Design department who know SQL

Department	Specialisation	Name	Salary
Design	SQL	Rehan D	150000
	SQL	Rai Y	100000

Part(f): Find all people who know SQL irrespective of department

Department	Name	Salary
Database	Ravi M	200000
Design	Rehan D	150000
Design	Rai Y	100000

Part(g): Find mean Salary of employee of each department	
	Salary
Department	
Database	200000.000000
Design	116666.666667

Figure 7: Output of Program of Figure 6

Please note the following points about the Program of Figure 6 and its output (Figure 7):

- Part (a) of Figure 6 creates the data frame **employeedata** consisting of data, as shown in Figure 1.
- Part (b) of Figure 6 creates a hierarchical index on attributes Department and Specialisation. It prints this index, which is shown along with the entire data in Part (b) of the output (Figure 7). You may please note that the indexed data is not in sorted order.
- Part (c) of Figure 6 sorts the data of the hierarchical index. This can be seen in Part (c) of the output. You may notice that the data is sorted in both Department and Specialisation.
- Part (d) of Figure 6 shows data values related to the Database Department having a specialisation in SQL, which is shown in Part (d) of Figure 7.
- Part (e) of Figure 6 demonstrates the slicing operation using `.loc` method; the results of this method are shown in Part (e) of Figure 7.
- Part (f) of Figure 6 demonstrates the use of swapping operation, which swaps the levels of the index, to answer this query, first the department and specialisation levels of index are swapped, resulting in Specialisation, Department hierarchical index. This is followed by a `.loc` method to display employees who know SQL irrespective of department. This is possible as index on Specialisation now is at higher level. You may please note the result given in Part (f) of Figure 7.
- Finally, Part(g) of Figure 6 demonstrates the use of grouping by Department level in hierarchical index of Department, Specialisation. The results of this is shown in Part(g) of Figure 7.

4.3.1 Reordering Index Levels

Reordering and sorting are two important operations on index levels of a hierarchical index. Reordering allows swapping of levels of indexes. For example, in Program 6, we have created a hierarchical index on Department and within department Specialisation. This is one of the logical hierarchy, as most of the queries would require grouping of data on Department and within department on Specialisation. However, in certain cases you may want to reverse the order. For example, in Part(f) of Figure 6, we want to find employees who are specialised in SQL. However, the index on Department, Specialisation will not give efficient answer. However, changing the levels as Specialisation, Department will group all the employees with a specialisation in a same group. Thus, enabling efficient data access.

Reordering Hierarchical Index Using Pandas

Reordering of index is done using `swaplevel()` method. This method swaps the level of two indexing attributes. The format is:

SwappedIndexName = ExistingIndexName.swaplevel('L1', 'L2')

The method is called by an existing hierarchical index consisting of hierarchy levels as L1, L2. The SwappedIndexName will get the swapped hierarchical levels as L2, L1. An example of use of this method is shown in Part (f) of Figure 6 and the result of this method is shown in Part (f) of Figure 7.

4.3.2 Sorting Levels

In general, when you create a hierarchical index, it does not sort the data frames, which remain in the order of the default index, which starts from value 0, as shown in Figure 1. Sorting on levels allows sorting of data frames as per the hierarchical levels of an index. For example, in Program 6 Part (b), we have created a hierarchical index on Department and within the department Specialisation. However, when we use this index to display the data frame, it is displayed as Part (b) of Figure 7. You may check that order of display is same as that of default index, as shown in Figure 1. This hierarchical index is then sorted, as shown in Part (c) of Figure 6, and now the data frame is printed using the index. You may please note that after sorting the index, the data frame is displayed as shown in Part (c) of Figure 7. You may observe that all the department names are now being displayed in sorted order, and within each department, the specialisation is also being displayed in sorted order. Sorting operation facilitates grouping of data and hence results in better performance of aggregate functions, if performed on sorted groups.

Sorting Hierarchical Index Using Pandas

Reordering of index is done using `sort_index()` method. The format of this method is:

`SortedIndexName = ExistingIndexName.sort_index()`

The method is called in an existing hierarchical index and sorting is performed. An example of use of this method is shown in Part (c) of Figure 6 and the result of this method is shown in Part (c) of Figure 7.

Check Your Progress 1

Answer the following questions for the data given in the following table:

Student Result in Courses			
Index #	Student Name	Course	Grade
0	X	MCS211	A
1	X	MCS212	B
2	X	MCS213	A
3	Y	BCS201	C
4	Y	BCS202	C
5	Z	MCS062	A
6	Z	MCS211	B
7	Z	MCS063	B

Programme Registration	
Student Name	Programme
X	MCA
Y	BCA
Z	MSC
Alpha	BCA
Beta	MCA

1. Create a data frame for the *Student Results in courses* and create a sorted index on *Course* for the data frame.

.....
.....
.....

2. Use the index to display the results of the course MCS211.

.....
.....
.....

3. Create a Hierarchical index on Student Name and Course, and display the results if the sorted index.

.....
.....
.....

4. Find the Result of the student named Z in the MCS211 course using the index made in question 3.

.....

5. Display the details of the student whose name is X.

.....

6. Display the course-wise results of all the students.

.....

4.4 COMBINING AND MERGING DATASETS

The hierarchical indexing, as discussed in the previous session, is used to organise data using different indexes. However, data from different sources is often combined to create a single, consistent dataset. This process is termed as combining the Datasets. You may please note that you can combine two datasets if some key attribute, index, or attribute is common to both datasets. In case you are using the Pandas library of Python, then the combining operation is performed on objects having a structure similar to a series or a data frame. In this section, we discuss the two important data wrangling processes, namely merging and concatenating.

4.4.1 Merging

Merging combines two data frames, which have at least one common key attribute. The objective of merging is to combine data from two different data frames into a single logical data frame that contains merged data of the two data frames based on the identical value of the merged key in the two data frames. You may please note that the logic of merging two data frames is almost identical to that of the relational join operation. Merging can be performed on one or more keys. We will demonstrate merging with the help of an example.

Consider the data of employees of an organisation as given in Figure 1. Consider that the organisation also stores the data of the Projects on which each employee is working. A separate data frame is being maintained to keep this information. Figure 8 shows this data.

Name	Project
Arav S	XYZ Web
Arav S	Ecommerce Books
Ravi M	Ecommerce Books
Rehan D	XYZ Web
Ben A	ABC Web

Figure 8: The Project Data of Employees given in Figure 1

You may please note that 'Rai Y' is a new employee and has not been assigned to any project, whereas 'Arav D' is working on two projects. Please also note that we are assuming that the Name of an employee is unique. In order to give the details of the employee and the projects on which they are working, we may combine the data frames that can be created for the data in Figures 1 and 8. Program 3, given in Figure 9, does this task.

```
#Program 3                                Part (a)
import pandas as pd
Figure1 = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
                  'Rai Y'],
          'Department': ['Design', 'Database', 'Design', 'Database',
                        'Design'],
          'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
          'Salary': [100000, 200000, 150000, 200000, 100000]}
employeeedata = pd.DataFrame(Figure1)
Figure8={'Name': ['Arav S', 'Arav S', 'Ravi M', 'Rehan D', 'Ben
A'], 'Project': ['XYZ Web', 'Ecommerce Books', 'Ecommerce
Books', 'XYZ Web', 'ABC Web']}
employeeeproject = pd.DataFrame(Figure8)
```

```
#Program 3                                Part (b)
print("Part (b): Index on 'Name' on the data of Figure 1")
Employeeedata_name_index = employeeedata.set_index(['Name'])
print(Employeeedata_name_index)
print("\n")
print("Part (b): Index on 'Name' on the data of Figure 8")
Employeeedata_name_indexonProject=
employeeeproject.set_index(['Name'])
print(Employeeedata_name_indexonProject)
print("\n")
```

```
#Program 3                                Part (c)
print("Part (c): Merging of the two data frames on the key
'Name'")
MergedData =
pd.merge(Employeeedata_name_index, Employeeedata_name_indexonPro
ject, on=['Name'])
print(MergedData)
print("\n")
```

```
#Program 3                                Part (d)
print("Part (d): Merging of data frames on the key 'Name' using
Left Join")
MergedLeftData =
pd.merge(Employeeedata_name_index, Employeeedata_name_indexonPro
ject, how='left', on=['Name'])
print(MergedLeftData)
print("\n")
```

Figure 9: Program for merging the data frames on a key

The output of Figure 9 is shown in Figure 10.

```
Part (b): Index on 'Name' on the data of Figure 1
      Department  Specialisation  Salary
Name
Arav S      Design  Web Development  100000
Ravi M    Database          SQL    200000
Rehan D      Design          SQL    150000
Ben A      Database  Web Development  200000
Rai Y      Design          SQL    100000
```

```
Part (b): Index on 'Name' on the data of Figure 8
      Project
Name
Arav S      XYZ Web
Arav S    Ecommerce Books
Ravi M    Ecommerce Books
Rehan D      XYZ Web
Ben A      ABC Web
```

Part (c): Merging of the two data frames on the key 'Name'				
Name	Department	Specialisation	Salary	Project
Arav S	Design	Web Development	100000	XYZ Web
Arav S	Design	Web Development	100000	Ecommerce Books
Ravi M	Database	SQL	200000	Ecommerce Books
Rehan D	Design	SQL	150000	XYZ Web
Ben A	Database	Web Development	200000	ABC Web

Part (d): Merging of data frames on the key 'Name' using Left Join				
Name	Department	Specialisation	Salary	Project
Arav S	Design	Web Development	100000	XYZ Web
Arav S	Design	Web Development	100000	Ecommerce Books
Ravi M	Database	SQL	200000	Ecommerce Books
Rehan D	Design	SQL	150000	XYZ Web
Ben A	Database	Web Development	200000	ABC Web
Rai Y	Design	SQL	100000	NaN

Figure 10: Output of the Program of Figure 9 showing merging

Please observe the following points about Figure 9 and Figure 10.

1. In Part (a) of Figure 9, the first statement imports the classes and methods of the pandas library of Python. This is followed by creation of two data frames one from the data of Figure 1 and second from the data of Figure 8. The names of these data frames are `employee_data` and `employee_project` respectively.
2. In Part (b) of Figure 9, prints the two name indexes. The first index is named `Employee_name_index` and is created on 'Name' attribute of `employee_data` data frame, while second index is named `Employee_data_name_index` on `Project`, which is also created on 'Name' attribute. Please note that 'Name' attribute exists in both the data frames and is defined over the same domain. You may observe the result of code of Part(b) of Figure 9 in Figure 10.
3. You may notice that the employee named 'Arav S' works on two projects namely 'XYZ Web' and 'Ecommerce Books'.
4. Please note that Part (c) of Figure 9 merges the two data frames on the 'Name' attribute or column. You may observe the result of this merge in part (c) of Figure 10. Observe that as the employee 'Rai Y' has not been assigned to any project so far, his name does not appear in the output. You may compare this merge operation with join operation of relational database management system.
5. Please also note that in Part (c) of Figure 10, since 'Arav S' has been assigned to two projects, his information appears for both the project. This is also similar to join operation of relational database management system.
6. Please note that Part (d) of Figure 9 merges the two data frames on the 'Name' attribute and uses `how='left'`. You may observe the result of this merge in part (d) of Figure 10. Observe that as the employee 'Rai Y' has not been assigned to any project so far, however, his name appears in the output though project assignment is 'NaN' (not a number) that is he is not assigned to any project. You may compare this merge operation with left outer join operation of relational database management system (RDBMS).

Please note the use of `how='left'` clause in the Figure 9 Part (d). This ensures that all the rows of the left data frame, of the two data frames that are being merged, appear in the output. In case, a row does not have any matching index value of 'Name' with the right data frame index, then 'NaN' will appear in the columns contributed by the right data frame. You can also use `how='right'` and `how='outer'` options while merging, which are similar to right outer join of RDBMS and full outer join of RDBMS.

4.4.2 Concatenating

Concatenation is the process of adding two data frames or series objects either horizontally or vertically. Let us explain some of the basic ways of concatenating data.

For example, assuming that an organisation has a second office at a different location, which has only one Department -Database. Figure 11 shows the list of the employees of that office.

Index #	Name	Department	Specialisation	Salary
0	Priya Y	Database	SQL	150000
1	Pramod N	Database	SQL	100000
2	Ben A	Database	Web Development	200000

Figure 11: List of employees in the second office.

Now, you are required to create a single list of all the employees, then you will be using the concat() method. Program 4, as shown in Figure 12, shows an example of the use of the concat() method with different options for concatenation of rows, while Figure 13 shows the output of Figure 12.

```
#Program 4                                Part (a)
import pandas as pd
Figure1 = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
                  'Rai Y'],
          'Department': ['Design', 'Database', 'Design', 'Database',
                        'Design'],
          'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
          'Salary': [100000, 200000, 150000, 200000, 100000]}
employeeedata = pd.DataFrame(Figure1)
Figure11={'Name': ['Priya Y', 'Pramod N', 'Ben A'],
          'Department': ['Database', 'Database', 'Database'],
          'Specialisation': ['SQL', 'SQL', 'Web Development'],
          'Salary': [150000, 100000, 200000]}
employeeedata2 = pd.DataFrame(Figure11)
print(employeeedata)
print("\n")
print(employeeedata2)
print("\n")
```

```
#Program 4                                Part (b)
print("Part (b): Concatenating data of Figure 1 and Figure 11")
ListOfEmployee = pd.concat([employeeedata, employeeedata2])
print(ListOfEmployee)
print("\n")
```

```
#Program 4                                Part (c)
print("Part (c): Concatenating data of Figure 1 and Figure 11
No Duplicate Data Frame and Resetting the Index")
ListOfEmployeeNoDup = pd.concat([employeeedata,
employeeedata2]).drop_duplicates().reset_index(drop=True)
print(ListOfEmployeeNoDup)
print("\n")
```

Figure 12: Program of Concatenation of data of Figure 1 and Figure 11

	Name	Department	Specialisation	Salary
0	Arav S	Design	Web Development	100000
1	Ravi M	Database	SQL	200000
2	Rehan D	Design	SQL	150000
3	Ben A	Database	Web Development	200000
4	Rai Y	Design	SQL	100000

	Name	Department	Specialisation	Salary
0	Priya Y	Database	SQL	150000
1	Pramod N	Database	SQL	100000
2	Ben A	Database	Web Development	200000

Part (b): Concatenating data of Figure 1 and Figure 11				
	Name	Department	Specialisation	Salary
0	Arav S	Design	Web Development	100000
1	Ravi M	Database	SQL	200000
2	Rehan D	Design	SQL	150000
3	Ben A	Database	Web Development	200000
4	Rai Y	Design	SQL	100000
0	Priya Y	Database	SQL	150000
1	Pramod N	Database	SQL	100000
2	Ben A	Database	Web Development	200000

Part (c): Concatenating data of Figure 1 and Figure 11 No Duplicate Data Frame and Resetting the Index				
	Name	Department	Specialisation	Salary
0	Arav S	Design	Web Development	100000
1	Ravi M	Database	SQL	200000
2	Rehan D	Design	SQL	150000
3	Ben A	Database	Web Development	200000
4	Rai Y	Design	SQL	100000
5	Priya Y	Database	SQL	150000
6	Pramod N	Database	SQL	100000

Figure 13: Output of Concatenation of data of Figure 1 and Figure 11

Please observe the following points about Figure 12 and Figure 13:

1. Part (a) of Figure 12 creates two data frames based on the data of Figures 1 and 11 with the names `employeedata` and `employeedata2`, respectively. The output is shown in the top half of Figure 13.
2. In Part (b) of Figure 12, two data frames were concatenated using `concat()` method into the variable `ListOfEmployee`. The concatenated list is shown in Figure 13 (b). You may please note that this list contains all the rows of both the data frames. Further, you may observe that this output has a duplicate row containing the name 'Ben A'.
3. The `concat` method of Part (c) followed by methods `drop_duplicates()` and `reset_index(drop=True)` ensures that duplicates are dropped and the index is reset (you may check the index # in Part (c) of Figure 13).

As far as horizontal concatenation is concerned, we perform it on the same data but with a reduced number of columns, as shown in Program 5 given in Figure 14.

```
#Program 5 Part (a)
import pandas as pd
Figure1part = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A', 'Rai Y'],
               'Department': ['Design', 'Database', 'Design', 'Database', 'Design']}
employeedatapart = pd.DataFrame(Figure1part)
Figure11part={'Specialisation': ['SQL', 'SQL', 'Web Development'], 'Salary': [150000, 100000, 200000]}
employeedata2part = pd.DataFrame(Figure11part)
print(employeedatapart)
print("\n")
print(employeedata2part)
print("\n")
```

```
#Program 5 Part (b)
print("Part (b): Concatenating part data of Figure 1 and Figure 11")
ListOfEmployee =
pd.concat([employeedatapart, employeedata2part], axis = 1)
print(ListOfEmployee)
print("\n")
```

Figure 14: Program 5 for Concatenating part data of Figure 1 and Figure 11

	Name	Department		
0	Arav	S	Design	
1	Ravi	M	Database	
2	Rehan	D	Design	
3	Ben	A	Database	
4	Rai	Y	Design	
		Specialisation	Salary	
0		SQL	150000	
1		SQL	100000	
2	Web Development		200000	

Part (b): Concatenating part data of Figure 1 and Figure 11				
	Name	Department	Specialisation	Salary
0	Arav	S	Design	SQL 150000.0
1	Ravi	M	Database	SQL 100000.0
2	Rehan	D	Design	Web Development 200000.0
3	Ben	A	Database	NaN NaN
4	Rai	Y	Design	NaN NaN

Figure 15: Output of Figure 14

Please observe the following points about Figure 14 and Figure 15:

1. Part (a) of Figure 14 creates two data frames based on the data of parts of Figures 1 and 11 with the names `employee_data_part` and `employee_data2_part`, respectively. The output is shown in the top half of Figure 15.
2. In Part (b) of Figure 14, the two data frames are concatenated horizontally using the `concat()` method with an option of `axis = 1` (indicating Horizontal concatenation). Please note that the first data frame has Name and Department columns, whereas the second data frame has Specialisation and Salary columns. Further, the number of rows in the first data frame is 5, whereas in the second data frame it is 3. Thus, the concatenated data frame contains NaN in the Specialisation and Salary columns of the 4th and 5th row (index# 3 and 4, respectively).

The horizontal concatenation may be very useful when the data is available in two data frames. In the next sub-section, we discuss the ways of combining the data frames.

4.4.3 Combining Data with Overlap

Combining data frames is a very interesting way of combining the data from two different sources. It differs from merging and concatenating, as it tries to combine missing data in a row with available data in another file. Consider an example, suppose the data of employees of an organisation (Figure 1) got corrupted. The file was recovered, but it lost part of the data in a few columns. An old backup of this file was found and is used to recover the data of the file. Please note that such recovery will not guarantee complete correctness. We are showing the process of performing such actions using Pandas of Python.

Figure 16 shows the program for combining two files using Pandas, while Figure 17 shows the output of the Program of Figure 16.

```
#Program 6: Part (a)
#Import the Pandas library
import pandas as pd
import numpy as np
#Create a New Employee data from Part of Data of Figure 1
Figure1New = {'Name': ['Arav S', 'Ravi M', 'Ben A', 'Rai Y'],
              'Department': ['Design', np.nan, np.nan, 'Design'],
              'Specialisation': [np.nan, 'SQL', 'Web Development',
                                np.nan],
              'Salary': [100000, 200000, 200000, 100000]}
```

```

employeeDataNew = pd.DataFrame(Figure1New)
#Create the OLD employee data Frame without Salary data
Figure1Old = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
'Rai Y'],
'Department': ['Design', 'Database', 'Design', 'Database',
'Design'],
'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL']}
employeeDataOld = pd.DataFrame(Figure1Old)
# Part (a): Create sorted index on 'Name' for both the data
frames
print("Part(a): Create sorted index on 'Name' for both the data
frames")
EmployeeData_New_sorted =
employeeDataNew.set_index(['Name']).sort_index()
print("Sorted Index on Name for New Employee data")
print(EmployeeData_New_sorted)
EmployeeData_Old_sorted =
employeeDataOld.set_index(['Name']).sort_index()
print("\n")
print("Sorted Index on Name for Old Employee data")
print(EmployeeData_Old_sorted)
print("\n")

```

```

# Part (b): Combining data using Combine.first method
print("Part(b): Combining data using Combine.first method")
finalList =
EmployeeData_New_sorted.combine_first(EmployeeData_Old_sorted)
print(finalList)
print("\n")

```

Figure 16: Program to combine two data frames

Part(a): Create sorted index on 'Name' for both the data frames

Sorted Index on Name for New Employee data

Name	Department	Specialisation	Salary
Arav S	Design	NaN	100000
Ben A	NaN	Web Development	200000
Rai Y	Design	NaN	100000
Ravi M	NaN	SQL	200000

Name	Department	Specialisation	Salary
Arav S	Design	NaN	100000
Ben A	NaN	Web Development	200000
Rai Y	Design	NaN	100000
Ravi M	NaN	SQL	200000

Sorted Index on Name for Old Employee data

Name	Department	Specialisation
Arav S	Design	Web Development
Ben A	Database	Web Development
Rai Y	Design	SQL
Ravi M	Database	SQL
Rehan D	Design	SQL

Name	Department	Specialisation
Arav S	Design	Web Development
Ben A	Database	Web Development
Rai Y	Design	SQL
Ravi M	Database	SQL
Rehan D	Design	SQL

Part(b): Combining data using Combine.first method

Name	Department	Salary	Specialisation
Arav S	Design	100000.0	Web Development
Ben A	Database	200000.0	Web Development
Rai Y	Design	100000.0	SQL
Ravi M	Database	200000.0	SQL
Rehan D	Design	NaN	SQL

Figure 17: Results of combining two data frames

Please observe the following points about Figure 16 and Figure 17:

1. Part (a) of Figure 16 creates two data frames. The first data frame, named `employeeDataNew`, has some missing values in different attributes (Please refer to Figure 17, in which NaN represents the missing values). Further, one row containing the name 'Rehan D' is not in the New data. The second data frame, named `employeeDataOld`, does not have the Salary column. Both the data

frames are sorted on the 'Name' column. The two sorted data frames are shown in the top half of Figure 17.

2. In Part (b) of Figure 16, the two sorted data frames are combined using `combine.first()` method. The result of this method is shown in Figure 17, part (b). Please note that all the missing values of the New data frames are replaced by old values from the Old data frame. Please also observe that the row containing the name 'Rehan D' has also been added; however, the Salary information of this row is not available.

In the next section, we discuss methods for reshaping data.

Check Your Progress 2

Answer the following questions for the data given in the table of Check Your Progress

1. Additionally, for questions 3 and 4, you may use the following supplementary data.

	StudentName	Course	Grade
0	X	MCS211	A
1	Y	BCS202	C
2	Z	MCS213	A
3	Alpha	BCS201	C
4	Beta	BCS202	C

1. Display the results of the students after including the programme name.

.....
.....
.....

2. Display the names of all the students, irrespective of whether they have appeared in any examination or not.

.....
.....
.....

3. It was noted that another file of data exists, which contains the result data as shown above. Concatenating the data of the two files.

.....
.....
.....

4. Concatenate the two files after removing the duplicate data.

.....
.....
.....

4.5 RESHAPING

Reshaping is the process of arranging the data differently from the standard two-dimensional tables, consisting of rows, which represent the information of an object, and columns, which represent attributes or features or variables of the data. Reshaping helps in looking at the data from a different perspective. In this section, we discuss two important ways of reorganising data, namely, pivoting and stacking & unstacking.

4.5.1 Pivoting

Pivoting is a method of organising data using more features, so that they can be understood in a better way. This also facilitates the identification of patterns and basic summarisation. Let us explain this with the help of an example. Consider the data of Figure 1 (only four records). This data can be organised using the two dimensions 'Department' and 'Specialisation', which is demonstrated with the help of the following program.


```
#Program 7: Use of Pivoting
import pandas as pd
# Create the DataFrame
Figure1 = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A'],
           'Department': ['Design', 'Database', 'Design', 'Database'],
           'Specialisation': ['Web', 'SQL', 'SQL', 'Web'],
           'Salary': [100000, 200000, 150000, 200000]}
employeedata = pd.DataFrame(Figure1)
print(employeedata)
print("\n")
```

```
print("Pivot on Department and Specialisation values of Name")
employee_Pivot=employeedata.pivot(index='Department',
columns='Specialisation', values='Name')
print(employee_Pivot)
print("\n")
```

```
print("Pivot on Department and Specialisation values Name and Salary")
employee_Pivot=employeedata.pivot(index='Department',
columns='Specialisation', values=['Name', 'Salary'])
print(employee_Pivot)
```

Figure 18: Pivoting using Department and Specialisation

	Name	Department	Specialisation	Salary
0	Arav S	Design	Web	100000
1	Ravi M	Database	SQL	200000
2	Rehan D	Design	SQL	150000
3	Ben A	Database	Web	200000

Pivot on Department and Specialisation values of Name			
Specialisation	SQL	Web	
Department			
Database	Ravi M	Ben A	
Design	Rehan D	Arav S	

Pivot on Department and Specialisation values Name and Salary				
	Name		Salary	
Specialisation	SQL	Web	SQL	Web
Department				
Database	Ravi M	Ben A	200000	200000

Figure 19: Result of Pivoting on Department and Specialisation

Please observe the following points about Figure 18 and Figure 19:

1. The first part of Figure 18 creates a data frame having the name employeedata. It has just four rows, as shown in the top part of Figure 19.
2. Next, a pivot is created on the data using the pivot () method. You may please observe the parameters used in the pivot method: the *index* specifies the row, the *columns* specify the column of the pivot, and the *values* specify the data values to be listed. Please observe the result of the pivot table.
3. In the final part of program 7, the pivot table has the same index and columns, and two values are specified. The result of the code is shown in the last part of Figure 19.

4.5.2 stack() and unstack() methods

Stack and unstack methods are used to change the record organisation. This is explained with the help of the following program.

```
#Program 8: Part (a)
#Import the Pandas library
import pandas as pd
#Create the data Frame
```

```
Figure1 = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',  
                  'Rai Y'],  
          'Department': ['Design', 'Database', 'Design', 'Database',  
                        'Design'],  
          'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web  
Development', 'SQL'],  
          'Salary': [100000, 200000, 150000, 200000, 100000]}  
employeedata = pd.DataFrame(Figure1)
```

```
# Part (b): Reshaping data using Stack method  
print("Part (b): Reshaping data using Stack method ")  
stackedEmployeeData = employeedata.stack()  
print(stackedEmployeeData)  
print("\n")
```

```
# Part (c): Reshaping data using unstack method  
print("Part (c): Reshaping data using unstack method")  
UnstackedEmployeeData = stackedEmployeeData.unstack()  
print(UnstackedEmployeeData)
```

Figure 20: Use of stack() and unstack() methods

Part (b): Reshaping data using Stack method

```
0  Name      Arav S  
   Department    Design  
   Specialisation Web Development  
   Salary      100000  
1  Name      Ravi M  
   Department    Database  
   Specialisation SQL  
   Salary      200000  
2  Name      Rehan D  
   Department    Design  
   Specialisation SQL  
   Salary      150000  
3  Name      Ben A  
   Department    Database  
   Specialisation Web Development  
   Salary      200000  
4  Name      Rai Y  
   Department    Design  
   Specialisation SQL  
   Salary      100000  
dtype: object
```

Part (c): Reshaping data using unstack method

```
   Name Department Specialisation Salary  
0  Arav S      Design Web Development 100000  
1  Ravi M      Database          SQL 200000  
2  Rehan D      Design          SQL 150000  
3  Ben A      Database Web Development 200000  
4  Rai Y      Design          SQL 100000
```

Figure 21: Result of use of stack() and unstack() methods

Please observe the following points about Figure 20 and Figure 21:

1. The first part of Figure 20 creates a data frame named employeedata. The data is the same as shown in Figure 1.
2. Next, the employee data object calls the stack() method in Figure 20, part (b). This result is a stacked display of data, as shown in Figure 21, part (b).
3. In part (c) of Figure 20, the stacked data object calls the method to unstack. This unstacks the data, which is displayed in Figure 21, part (c).

4.5.3 Reshaping with Hierarchical Indexing

The use of Hierarchical Indexing has been defined in Section 4.3. Here, we demonstrate the use of hierarchical or multidimensional indexing in the reshaping of data with the help of an example program.

```
#Program 9Part (a)
import pandas as pd
employee = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
                    'Rai Y'],
            'Department': ['Design', 'Database', 'Design', 'Database',
                           'Design'],
            'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
            'Salary': [100000, 200000, 150000, 200000, 100000],
            'Project': ['XYZ Web', 'Ecommerce', 'Ecommerce', 'XYZ
Web', 'XYZ Web']}
employeedata = pd.DataFrame(employee)
```

```
# Part (b): Creating an index
Employeedata_multi_index_sorted =
employeedata.set_index(['Department', 'Specialisation',
                        'Project']).sort_index()
print("Creating and Displaying Multi Sorted Index")
print(Employeedata_multi_index_sorted)
print("\n")
```

```
# Part (c): Swapping Department with Project index levels
print("Reshaping Data Project First")
Employeedata_multi_index_sortedandswapped =
Employeedata_multi_index_sorted.swaplevel('Project',
                                           'Department').sort_index()
print(Employeedata_multi_index_sortedandswapped)
print("\n")
```

Figure 22: Reshaping data using hierarchical indexing

Creating and Displaying Multi Sorted Index				
Department	Specialisation	Project	Name	Salary
Database	SQL	Ecommerce	Ravi M	200000
	Web Development	XYZ Web	Ben A	200000
Design	SQL	Ecommerce	Rehan D	150000
		XYZ Web	Rai Y	100000
	Web Development	XYZ Web	Arav S	100000

Reshaping Data Project First				
Project	Specialisation	Department	Name	Salary
Ecommerce	SQL	Database	Ravi M	200000
		Design	Rehan D	150000
XYZ Web	SQL	Design	Rai Y	100000
	Web Development	Database	Ben A	200000
		Design	Arav S	100000

Figure 23: Output of reshaping data using hierarchical indexing

Please observe the following points about Figure 22 and Figure 23:

1. The first part of Figure 22 creates a data frame named `employeedata`. This data has one more column, 'Project'. This has been created to show the reshaping of the hierarchical index with multiple columns.
2. In part (b) of Figure 22, a sorted index is created on Department, Specialisation and Project. This index is shown in Figure 23.
3. In part (c) of Figure 22, the data is reshaped by swapping the Department level with the Project level. The reshaped index is then sorted. The result of this reshaping is shown in Figure 23.

Check Your Progress 3

1. What is the use of pivoting?

2. What is the purpose of stack and unstack methods?

3. How is hierarchical indexing used for reshaping?

4.6 SUMMARY

This unit provides details on the following aspects of data combining and reshaping. The unit also illustrates the concepts with the help of examples of Python code.

1. Purpose of Indexing in a data frame
2. The process of creating an index using pandas library of Python
3. The purpose and advantages of using hierarchical indexing
4. Creating a hierarchical index using pandas library of Python
5. The coding of reordering and sorting levels of indexes for flexible data processing.
6. In the topic of Combining datasets, the coding of merging of data sets was discussed. In addition, the coding of concatenating the datasets were explained.
7. The use of reshaping the data sets were defined and Python coding was explained for the same.

4.7 SOLUTIONS/ANSWERS

Check Your Progress 1

```
import pandas as pd
studentdata = {'StudentName': ['X', 'X', 'X', 'Y', 'Y', 'Z',
                                'Z', 'Z'], 'Course': ['MCS211', 'MCS212', 'MCS213', 'BCS201',
                                'BCS202', 'MCS062', 'MCS211', 'MCS063'],
               'Grade': ['A', 'B', 'A', 'C', 'C', 'A', 'B', 'B']}
studentdataframe = pd.DataFrame(studentdata)
```

```
# Answer to CYP 1 Question 1 Creating an index
Studentdata_index_sorted =
studentdataframe.set_index(['Course']).sort_index()
# Answer to CYP 1 Question 1 Displaying the indexed data
print("Result of CYP 1 Question 1 \nDisplaying Sorted Index")
print(Studentdata_index_sorted, "\n")
```

```
# Answer to CYP 1 Question 2 Displaying the Result of MCS211
print("Result of CYP 1 Question 2 \nDisplaying Result of
Students in MCS211")
print(Studentdata_index_sorted.loc['MCS211'], "\n")
```

```
# Answer to CYP 1 Question 3 Creating an Index on Student Name
and Course
Studentdata_multi_sorted_index =
studentdataframe.set_index(['StudentName',
'Course']).sort_index()
print("Result of CYP 1 Question 3 \nCreating and Displaying
Multi Index Sorted as asked")
print(Studentdata_multi_sorted_index, "\n")
```

```
# Answer to CYP 1 Question 4 Accessing data from Multi-index
print("Result of CYP 1 Question 4 \nFind the Result of student
named Z in MCS211 course")
print(Studentdata_multi_sorted_index.loc[('Z', 'MCS211')],
"\n")
```

```
# Answer to CYP 1 Question 5 Slicing data from Multi-index
print("Result of CYP 1 Question 5 \nFind the data of the
Student X only")
print(Studentdata_multi_sorted_index.loc[('X')], "\n")
```

```
# Answer to CYP 1 Question 6 Swapping Student Name with Course
print("Result of CYP 1 Question 6 \nFindCoursewise result of
students")
Studentdata_multi_sorted_indexandswapped =
Studentdata_multi_sorted_index.swaplevel('Course',
'StudentName').sort_index()
print(Studentdata_multi_sorted_indexandswapped, "\n")
```

Result of CYP 1 Question 1
Displaying Sorted Index

Course	StudentName	Grade
BCS201	Y	C
BCS202	Y	C
MCS062	Z	A
MCS063	Z	B
MCS211	X	A
MCS211	Z	B
MCS212	X	B
MCS213	X	A

Result of CYP 1 Question 2
Displaying Result of Students in MCS211

Course	StudentName	Grade
MCS211	X	A
MCS211	Z	B

```
Result of CYP 1 Question 3
Creating and Displaying Multi Index Sorted as asked
Grade
StudentName Course
X             MCS211      A
              MCS212      B
              MCS213      A
Y             BCS201      C
              BCS202      C
Z             MCS062      A
              MCS063      B
              MCS211      B
```

```
Result of CYP 1 Question 4
Find the Result of student named Z in MCS211 course
Grade      B
Name: (Z, MCS211), dtype: object
```

```
Result of CYP 1 Question 5
Find the data of the Student X only
Grade
Course
MCS211      A
MCS212      B
MCS213      A
```

```
Result of CYP 1 Question 6
FindCoursewise result of students
Grade
Course StudentName
BCS201 Y           C
BCS202 Y           C
MCS062 Z           A
MCS063 Z           B
MCS211 X           A
        Z           B
MCS212 X           B
MCS213 X           A
```

Check Your Progress 2

```
import pandas as pd
studentresult = {'StudentName': ['X', 'X', 'X', 'Y', 'Y', 'Z',
                                'Z', 'Z'], 'Course': ['MCS211', 'MCS212', 'MCS213', 'BCS201',
                                'BCS202', 'MCS062', 'MCS211', 'MCS063'],
                 'Grade': ['A', 'B', 'A', 'C', 'C', 'A', 'B', 'B']}
studentresultdataframe = pd.DataFrame(studentresult)
studentprogramme = {'StudentName': ['X', 'Y', 'Z', 'Alpha',
                                    'Beta'], 'Programme': ['MCA', 'BCA', 'MSC', 'BCA', 'MCA']}
studentprogdataframe = pd.DataFrame(studentprogramme)
```

```
print("CYP 2: Q1.Merging of the two data frames on the key
'StudentName'")
MergedData = pd.merge(studentprogdataframe,
studentresultdataframe, on=['StudentName'])
print(MergedData)
print("\n")
```

```
print("CYP 2: Q2. Merging of data frames on the key
'StudentName' using Left Join to display the names of all the
students, irrespective of whether they have appeared in any
examination")
MergedLeftData = pd.merge(studentprogdataframe,
studentresultdataframe, how='left', on=['StudentName'])
print(MergedLeftData)
print("\n")

studentresultadditional= {'StudentName': ['X', 'Y', 'Z',
'Alpha', 'Beta'], 'Course': ['MCS211', 'BCS202', 'MCS213',
'BCS201', 'BCS202'], 'Grade': ['A', 'C', 'A', 'C', 'C']}

Studentresultdataframe2 = pd.DataFrame(studentresultadditional)
```

```
print("CYP 2: Q3. Concatenating data of Two Results of the
students")
StudentResultConcat = pd.concat([studentresultdataframe,
Studentresultdataframe2])
print(StudentResultConcat)
print("\n")
```

```
print("CYP 2: Q4. Concatenating data of with No Duplicate
Data Frame and Resetting the Index")
Studentresultdataframe2NoDup = pd.concat
([studentresultdataframe,
Studentresultdataframe2]).drop_duplicates().reset_index(drop=True)
print(Studentresultdataframe2NoDup)
print("\n")
```

CYP 2: Q1. Merging of the two data frames on the key 'StudentName'

	StudentName	Programme	Course	Grade
0	X	MCA	MCS211	A
1	X	MCA	MCS212	B
2	X	MCA	MCS213	A
3	Y	BCA	BCS201	C
4	Y	BCA	BCS202	C
5	Z	MSC	MCS062	A
6	Z	MSC	MCS211	B
7	Z	MSC	MCS063	B

CYP 2: Q2. Merging of data frames on the key 'StudentName' using Left Join to display the names of all the students, irrespective of whether they have appeared in any examination

	StudentName	Programme	Course	Grade
0	X	MCA	MCS211	A
1	X	MCA	MCS212	B
2	X	MCA	MCS213	A
3	Y	BCA	BCS201	C
4	Y	BCA	BCS202	C
5	Z	MSC	MCS062	A
6	Z	MSC	MCS211	B
7	Z	MSC	MCS063	B
8	Alpha	BCA	NaN	NaN
9	Beta	MCA	NaN	NaN

Combining and Reshaping Datasets

CYP 2: Q3. Concatenating data of Two Results of the students

	StudentName	Course	Grade
0	X	MCS211	A
1	X	MCS212	B
2	X	MCS213	A
3	Y	BCS201	C
4	Y	BCS202	C
5	Z	MCS062	A
6	Z	MCS211	B
7	Z	MCS063	B
0	X	MCS211	A
1	Y	BCS202	C
2	Z	MCS213	A
3	Alpha	BCS201	C
4	Beta	BCS202	C

CYP 2: Q4. Concatenating data of with No Duplicate Data Frame and Resetting the Index

	StudentName	Course	Grade
0	X	MCS211	A
1	X	MCS212	B
2	X	MCS213	A
3	Y	BCS201	C
4	Y	BCS202	C
5	Z	MCS062	A
6	Z	MCS211	B
7	Z	MCS063	B
8	Z	MCS213	A
9	Alpha	BCS201	C
10	Beta	BCS202	C

Check Your Progress 3

1. Pivoting is done to organise data using two dimensions so that the data can be interpreted in a better way.
2. Stack methods organises data as attribute name-value pair, whereas unstack displays the data as a data frame.
3. Hierarchical indexing allows data to be displayed using multiple indexes, thus, can help in showing data in the structure of groups and sub-groups.